Linux `grep` Command Notes

👉**grep = Global Regular Expression Print**
👉Used to **search text patterns** inside files or command outputs.

---

1. Search for a Word in a File

```
grep "word" file.txt
```

Example: If `file.txt` contains:

```
This is a sample text file.
It contains multiple lines.
We are searching for a specific word.
```

```
grep "word" file.txt
```

Output:

```
We are searching for a specific word.
```

---

2. Case-Insensitive Search `-i`

```
grep -i "word" file.txt
```

Example:

```
Hello World
This is Linux
welcome to GREP
```

```
grep -i "grep" file.txt
```

Output:

```
welcome to GREP
```

---

3. Show Line Numbers `-n`

```
grep -n "Error" file.txt
```

Example:

```
Error: File not found
Warning: Low disk space
Error: Access denied
```

Output:

```
1:Error: File not found
3:Error: Access denied
```

---

4. Invert Match `-v`

```
grep -v "Error" file.txt
```

Example:

```
Success: Operation completed
Error: File not found
Warning: Low disk space
```

Output:

```
Success: Operation completed
Warning: Low disk space
```

---

5. Count Matches `-c`

```
grep -c "fail" file.txt
```

Example:

```
fail
success
```

```
fail
fail
```

Output:

```
3
```

---

6. Show Filenames with Matches `-l`

```
grep -l "Universe" *.txt
```

Example:

```
file1.txt: Hello World
file2.txt: Hello Universe
```

Output:

```
file2.txt
```

---

7. Recursive Search in Directories `-r`

```
grep -r "main" /project/src
```

Example:

```
/project/src/app.java:      public static void main(String[] args) {
/project/src/utils.java:    public static void main(String[] args) {
```

Explanation: Looks inside all files under `/project/src` and shows matches with filenames.

---

8. Match Whole Words Only `-w`

```
grep -w "cat" file.txt
```

Example:

```
The cat is on the roof.
The category is important.
```

Output:

```
The cat is on the roof.
```

---

9. Print Only the Matching Part `-o`

```
grep -o "foo[0-9]" file.txt
```

Example:

```
foo1 bar2 foo3 baz4
```

Output:

```
foo1
foo3
```

Another example:

```
grep -o "ID=[0-9]\+" log.txt
```

Output:

```
ID=101
ID=202
ID=303
```

---

10. Show Lines After Match `-A n`

```
grep -A 2 "Error" file.txt
```

Example:

```
Error occurred here
Line 3
Line 4
```

---

11. Show Lines Before Match `-B n`

```
grep -B 2 "Error" file.txt
```

Example:

```
Line 1
Line 2
Error occurred here
```

---

12. Show Lines Around Match `-C n`

```
grep -C 2 "Error" file.txt
```

Example:

```
Line 1
Line 2
Error occurred here
Line 4
Line 5
```

Summary Table

Option	Meaning	Explanation & Use Case	Example
`-i`	Case-insensitive search	Ignores case. Useful when you don't know the exact capitalization.	`grep -i "hello" file.txt`
`-n`	Show line numbers	Shows line number with each match.	`grep -n "error" logfile.log`
`-v`	Invert match (show non-matching lines)	Shows lines that do **not** match the pattern.	`grep -v "success" file.txt`

Option	Meaning	Explanation & Use Case	Example
`-c`	Count matches	Displays only the number of matches.	`grep -c "fail" file.txt`
`-l`	Show filenames with matches	Prints only filenames containing matches.	`grep -l "TODO" *.java`
`-r`	Recursive search in directories	Searches inside all files under a directory (recursively).	`grep -r "main" ./src`
`-w`	Match whole words only	Ensures only exact words match (not partial).	`grep -w "cat" file.txt`
`-o`	Print only the matching part	Prints just the matched part instead of the whole line.	`grep -o "foo[0-9]" file.txt`
`-A n`	Show n lines After match	Shows n lines after the match for context.	`grep -A 2 "Error" logfile.log`
`-B n`	Show n lines Before match	Shows n lines before the match for context.	`grep -B 2 "Error" logfile.log`
`-C n`	Show n lines Around match	Shows n lines before **and** after the match.	`grep -C 2 "Error" logfile.log`

Quick Examples

```
grep "error" logfile.log       # Search for "error" in logfile.log
grep -i "hello" file.txt       # Case insensitive
grep -n "error" logfile.log    # Show line numbers
grep -v "success" file.txt     # Show lines without "success"
grep -c "fail" file.txt        # Count matches
grep -l "TODO" *.java          # Show filenames with matches
grep -r "main" ./src           # Recursive search
grep -w "cat" file.txt         # Match whole word
grep -o "foo[0-9]" file.txt    # Show only matching part
grep -A 2 "Error" logfile.log   # Show 2 lines after match
grep -B 2 "Error" logfile.log   # Show 2 lines before match
grep -C 2 "Error" logfile.log   # Show 2 lines around match
```

👈 `grep` is one of the most powerful text-search tools in Linux, widely used in logs, codebases, and pipelines.